

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSESSMENT TOOL 1

Constraint-Based Problem Solving

Course Code: CSA15

Course Outcome: CO5 – Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.

Name : T . Kodanda Ram
Register Number : 192524162

Question 1 – HDFS-Based Storage Solution for a Media Platform

Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

1.1 Constraint and Requirement Analysis

Constraint	Design Priority	Response
Availability > 99.9%	Highest	Use redundant HDFS metadata services and distribute data replicas across independent failure domains.
Replication factor ≤ 3	High	Use a replication factor of 3 as the maximum for important datasets; do not exceed it.
Cost-efficient growth	High	Scale DataNodes incrementally, use tiered storage policies where appropriate, and avoid unnecessary replication.
Media workload	Functional	Store large media objects and metadata separately where useful, while using HDFS for scalable distributed storage.

1.2 Proposed Architecture

Media Uploads → Ingestion Layer → HDFS Client → Active/Standby NameNode → DataNodes across racks → Media Storage / Metadata → Analytics and downstream consumers

Architecture components:

HDFS NameNode services manage namespace and block metadata. A redundant NameNode arrangement supports metadata-service availability.

DataNodes store HDFS blocks and serve read/write requests.

A maximum replication factor of 3 provides three copies of configured replicated blocks without violating the constraint.

Rack-aware placement should distribute replicas across failure domains so that a rack-level failure does not remove every copy.

Additional DataNodes can be added as media volume increases, allowing incremental capacity expansion.

1.3 Working

When a media object is written, the HDFS client sends the file to HDFS, where the file is divided into blocks. The blocks are distributed across DataNodes. For datasets configured with replication factor 3, HDFS maintains up to three replicas. Rack-aware placement improves resilience by avoiding concentration of replicas in a single rack. As the platform grows, new DataNodes are added to increase aggregate capacity and throughput.

1.4 Constraint Satisfaction

Availability: redundant metadata services and distributed block replicas reduce the impact of individual node failures.

Replication limit: the design never exceeds a replication factor of 3.

Cost efficiency: capacity is expanded by adding DataNodes as demand grows rather than deploying excessive fixed capacity in advance.

Media scalability: distributed block storage allows large media collections to be spread across the cluster.

1.5 Justification

Replication factor 3 is selected because it is the highest value permitted by the question and provides stronger fault tolerance than lower replication for critical media data. Rack-aware placement is important because three copies located within one failure domain could still be lost together. Incremental DataNode expansion keeps the storage platform aligned with demand and avoids unnecessary initial infrastructure expenditure.

1.6 Conclusion

The proposed HDFS design prioritizes the required availability, respects the replication ceiling of three, and supports cost-efficient capacity growth. It therefore provides a robust and scalable storage foundation for the media platform.

Question 2 – MapReduce Workflow for Log Processing

A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

2.1 Constraint and Requirement Analysis

Constraint	Design Response
20-minute batch window	Use parallel map tasks, efficient key/value design, controlled reducer count, and avoid unnecessary data movement.
Limited cluster nodes	Use available resources carefully; size tasks to maximize node utilization without creating excessive task overhead.
Log processing	Parse records in Mapper, aggregate by key during Shuffle/Sort, and produce summarized results in Reducer.

2.2 Proposed Workflow

HDFS Log Files → Input Splits → Mapper Tasks → Shuffle and Sort → Reducer Tasks → HDFS Output

2.3 Map Function Design

Each Mapper reads log records from its assigned input split and emits compact intermediate key/value pairs.

Example logical output: (service, error_count) or (status_code, count). Filtering malformed records at the Mapper reduces unnecessary downstream traffic.

2.4 Reduce Function Design

Reducers receive grouped values for each key and calculate the required aggregate, such as total events, errors, or requests. The final output is written to HDFS for reporting or further processing.

2.5 Task Distribution

HDFS input is divided into input splits so multiple Mapper tasks can work in parallel.

Where possible, schedule map computation close to the data to reduce network transfer.

Use a reducer count appropriate to the limited cluster size; too many reducers can increase scheduling and shuffle overhead.

Partition intermediate keys so related records reach the same Reducer.

If a small number of nodes are available, prioritize balanced CPU, memory, and I/O utilization rather than creating an excessive number of concurrent tasks.

2.6 20-Minute Batch Strategy

The 20-minute requirement should be treated as a performance target. The job should use parallelism, data locality, compact intermediate records, efficient parsing, and an appropriate number of reducers. Before deployment, the actual input volume and cluster capacity must be benchmarked because the question does not provide a fixed data size or exact number of nodes; therefore an exact task count or guaranteed runtime cannot be established from the supplied information alone.

2.7 Justification

MapReduce is suitable because log records can be processed independently during mapping and aggregated by key during reduction. Keeping intermediate records compact reduces shuffle traffic, while balanced task distribution makes better use of limited nodes. The final configuration should be validated with representative workload measurements against the 20-minute target.

2.8 Conclusion

The proposed workflow uses parallel MapReduce processing while controlling task and reducer overhead. It directly addresses the 20-minute batch constraint and the limited-node constraint without claiming a runtime that cannot be verified from the assessment data.

Question 3 – YARN Scheduler Policy

A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

3.1 Constraint Analysis

Constraint	Policy
Fairness	Use separate queues/resource allocations so lower-priority workloads continue to receive a fair share.
ETL priority	Assign ETL a higher scheduling priority or higher queue capacity while preserving minimum resources for other workloads.
Node failure recovery	Use YARN resource management and application retry/recovery mechanisms so work can be rescheduled after failed-node detection.

3.2 Proposed Queue Structure

YARN ResourceManager → Scheduler → ETL Queue / Reporting Queue / ML Queue → NodeManagers → Containers

Queue	Priority	Policy Objective
ETL	High	Faster access to resources because ETL has explicit priority.
Reporting	Medium	Maintain predictable access without starving reporting

		workloads.
Machine Learning	Normal / workload-based	Receive fair resource access while allowing long-running jobs to use available capacity.

3.3 Scheduler Policy

Reserve a defined portion of cluster capacity for ETL so high-priority ETL jobs do not wait behind lower-priority workloads.

Use queue-level capacity or fair scheduling principles so reporting and ML retain access to resources.

Allow unused capacity to be borrowed by other queues when ETL demand is low, subject to the scheduler policy.

When a NodeManager becomes unavailable, YARN detects resource loss and running work can be rescheduled/retried according to application requirements.

Avoid absolute starvation of reporting and ML by enforcing minimum resource guarantees or fair-share limits.

3.4 Constraint Satisfaction

Fairness is achieved by preserving resources for all workload classes. ETL receives higher priority through queue priority/capacity. Node failure recovery is supported through YARN's resource management and task/application retry mechanisms. The exact numeric queue percentages should be configured after observing the organization's workload profile because no cluster capacity or workload ratios are specified in the question.

3.5 Justification

A single unrestricted priority queue could cause reporting and ML starvation. A multi-queue policy provides a better balance: ETL is favored as required, while minimum access for other workloads preserves fairness. YARN is appropriate because it separates cluster resource management from application execution and can respond to node/resource failures.

3.6 Conclusion

The policy combines ETL priority with fair resource access for reporting and ML and incorporates failure recovery through YARN-managed resource and application mechanisms.

Question4—ConstrainedEnterpriseData IngestionArchitecture

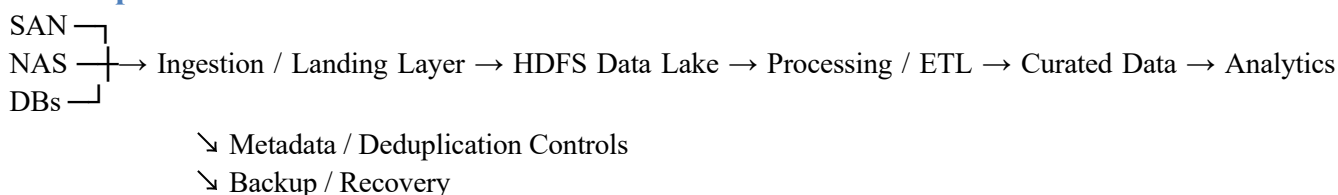
An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

4.1 Constraint Analysis

Constraint	Design Response
SAN ingestion	Use controlled ingestion processes to expose/copy required data into the big-data landing layer.
NAS ingestion	Use file-based ingestion with metadata and incremental identification where supported.
Operational databases	Use database-aware extraction, preferably incremental/CDC-style extraction when available.
Backup reliability	Maintain durable HDFS storage with appropriate replication and separate backup/recovery

	procedures.
Minimal duplication	Use source identifiers, checksums/hashes, metadata catalogs, and incremental ingestion rules.

4.2 Proposed Architecture



4.3 Component Roles

SAN connector/process: obtains authorized data from SAN storage without treating the SAN itself as the analytics platform.

NAS ingestion: transfers required files to a controlled landing area and tracks source metadata.

Database ingestion: extracts operational records using incremental changes where feasible to reduce repeated full copies.

Landing layer: provides a controlled boundary for validation, schema checks, and duplicate detection.

HDFS: provides distributed storage for the big-data platform.

Metadata/deduplication layer: records source system, object/file identifier, version, timestamp, and checksum where appropriate.

Backup/recovery layer: protects against storage or operational failures using a separate recovery strategy.

4.4 Minimal Duplication Strategy

Each incoming object or record should be associated with a source identifier and ingestion metadata. For files, a checksum can be used to detect identical content. For databases, source primary keys plus change timestamps/version information can identify already-ingested records. The pipeline should distinguish a new version from an exact duplicate rather than blindly creating another copy.

4.5 Backup Reliability

HDFS replication can protect against individual DataNode loss, but replication should not be treated as a complete independent backup strategy. Important enterprise data should also have a defined recovery copy or backup mechanism according to organizational policy.

4.6 Justification

Separating source-specific ingestion from common Hadoop storage makes the architecture easier to control and audit. Database-aware incremental ingestion reduces repeated transfer, while metadata and checksums help prevent exact duplicates. HDFS provides scalable distributed storage, while an independent recovery strategy addresses backup reliability.

4.7 Conclusion

The architecture integrates SAN, NAS, and operational database sources into a controlled Hadoop landing and storage platform while explicitly addressing duplicate detection and backup reliability.

Question 5 – End-to-End Hadoop Ecosystem Solution

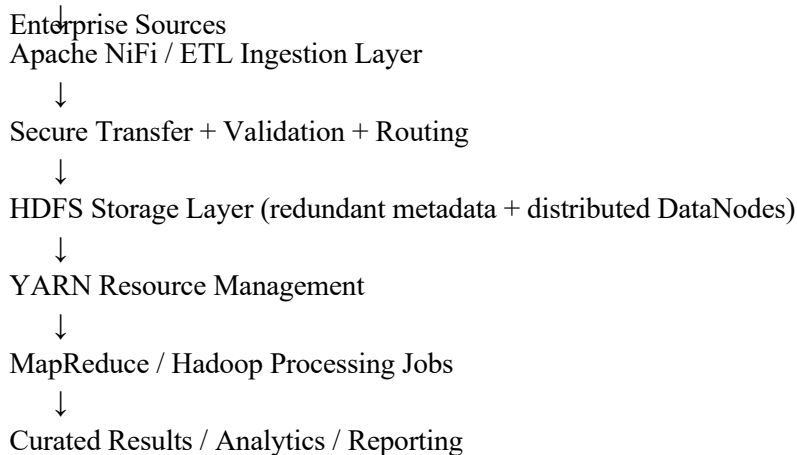
Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

5.1 Constraint Analysis

Constraint	Design Response
Secure data movement	Use authenticated and encrypted communication, controlled access, and secure pipeline configuration.
High availability	Use redundant critical services and distribute data/workloads to reduce single points of failure.
NiFi or ETL integration	Place Apache NiFi or an ETL layer between source systems and the Hadoop ingestion/storage services.
End-to-end Hadoop ecosystem	Integrate ingestion, HDFS storage, YARN resource management, processing, and downstream consumption.

5.2 Proposed End-to-End

Architecture



5.3 Secure Data Movement

Authenticate systems and users before allowing data transfer.

Use encrypted transport for data moving between trusted components where supported by the deployment.

Apply least-privilege access controls so ingestion services can write only to required locations.

Validate and audit transferred data through pipeline logs and metadata.

Protect credentials and secrets using the organization's approved secret-management approach rather than embedding them in pipeline definitions.

5.4 High Availability Design

Use redundant critical Hadoop control-plane services where supported so a single service failure does not stop the platform.

Distribute HDFS blocks across multiple DataNodes and failure domains.

Run ingestion components with appropriate redundancy so one failed pipeline instance does not stop all data movement.

Design processing applications for retry/recovery so transient node failures can be handled.

5.5 NiFi / ETL Integration

Apache NiFi can act as the controlled ingestion and routing layer. It can receive data from enterprise sources, validate or transform records where required, route them to HDFS, and maintain flow-level provenance. Alternatively, an ETL pipeline can perform equivalent extraction, transformation, validation, and loading functions. The assessment allows either NiFi or ETL integration.

5.6 End-to-End Data Flow

1. Source systems produce files, events, or database records.
2. NiFi/ETL authenticates the source and receives the data.
3. The pipeline validates format and metadata and applies required transformations.
4. Secure transfer sends approved data to HDFS.
5. HDFS stores the data across DataNodes with configured redundancy.
6. YARN allocates resources to processing applications.
7. MapReduce or other Hadoop processing jobs transform/aggregate the data.
8. Curated results are made available to downstream analytics or reporting systems.

5.7 Constraint Satisfaction

Constraint	How the design satisfies it
Secure data movement	Authentication, encryption, least privilege, validation, and auditing are built into the ingestion path.
High availability	Critical services are designed with redundancy; HDFS data is distributed across DataNodes/failure domains.
NiFi/ETL integration	NiFi/ETL is explicitly positioned as the ingestion and routing layer before HDFS.
End-to-end Hadoop	The flow covers ingestion → HDFS → YARN → processing → output.

5.8 Justification

Putting NiFi/ETL at the ingestion boundary separates source connectivity from Hadoop storage and processing. Security controls are concentrated around the data movement path, while redundant services and distributed HDFS storage reduce single points of failure. YARN provides centralized resource management for processing workloads. This architecture therefore addresses all constraints stated in the question without introducing unnecessary components.

5.9 Conclusion

The proposed end-to-end architecture integrates secure ingestion through NiFi/ETL, highly available Hadoop storage and resource management, and Hadoop processing. Each major component has a direct role in satisfying the stated constraints.